Field Hours

CSCE 708 Introduction to Computer Science Concepts and Programming

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Instructions run in the order you wrote

A program is a list of instructions the machine runs from top to bottom. In Java the list sits in a method named main. The statement int n = 4; stores 4 under the name n. The next line may use n. A nearby line does not run early. Order is the mechanism. Swap two legal lines and you have written a different program. Even a main of 4 lines has a first line and a last line.

You trace by simulating one line, then the next, on paper. Write the name n and the value 4. When a later line adds 1, cross out 4 and write 5. If your trace and the machine disagree at line 3, the bug is in an earlier line, not in the printer. Do not skip ahead. Four lines are few enough that you can finish the trace before you compile.

Run main from the top.

1. Enter main

2. Store 4 under the name n

3. Use n on the next line

4. Leave at the last line

Two stores, then a sum

1. Line 1 stores int a = 2. Line 2 stores int b = 2.

2. Line 3 stores int s = a + b. Your trace shows s equal to 4.

3. Line 4 prints s. The print shows 4 because the addition already happened on line 3, not inside the print.

Problem 1. Line 1 prints n, and line 2 stores int n = 4. What goes wrong when you run from the top?

Answer. Line 1 uses n before any store. Order is the mechanism: the store on line 2 has not run yet, so the print is not showing 4.

Problem 2. Your trace says s is 4 after int s = 2 + 2, and the program prints 5. Which line do you inspect?

Answer. The addition on the earlier line, not the print. The print shows whatever s holds. A trace that expects 4 means the store is where a printed 5 has to come from.

Lecture 2 · 90 minutes

A name, a type, and one stored value

A variable is a named slot with a type. In Java, int holds a whole number and double holds a fraction. You pick the type when you create the slot. The line int q = 7 / 2 stores 3, because both operands are ints and the remainder is dropped. The line double q = 7.0 / 2.0 stores 3.5. The operand types are the mechanism. A wider name on the left does not put the fraction back.

Assignment replaces the stored value. It does not keep a history. If n is 4 and you execute n = n + 1, the slot holds 5 and the 4 is gone. The right-hand side is read first, which is why n + 1 still sees 4. You must store a first value when you create a local, even when that value is 0. Reading a local you never stored is rejected at compile time.

Two divisions, two types.

int division

Operands 7 and 2

Remainder is dropped

Stored result is 3

double division

Operands 7.0 and 2.0

Fraction is kept

Stored result is 3.5

Update one slot twice

1. Create int n = 4. The slot holds 4.

2. Execute n = n + 1. The right-hand side reads 4, adds 1, and stores 5. The old 4 is gone.

3. Execute n = n + 1 again. The slot holds 6. Two updates from 4 give 6, not a list of older values.

Problem 1. What does int q = 7 / 2 store, and what does double q = 7.0 / 2.0 store?

Answer. The int division stores 3, because both operands are ints and the remainder is dropped. The double division stores 3.5.

Problem 2. n holds 4. You execute n = n + 1 twice. What is in the slot, and where did the original 4 go?

Answer. The slot holds 6. Each assignment replaces the previous value. The 4 was overwritten by 5, and the 5 was overwritten by 6.

Lecture 3 · 90 minutes

Tests, and, or, and one chosen block

A boolean is true or false. The test n > 0 is true when n is 3 and false when n is 0. if runs one block for true, and

else runs the other. You do not take both. && requires two tests. When n is 3 and m is 0, n > 0 && m > 0 is false because the second half fails. Java skips that second half when the first half of && is already false.

|| is true when either side is true. With n equal to 3 and m equal to 0, n > 0 || m > 0 is true. ! flips one boolean, so !(n > 0) is false when n is 3. Write the numbers into the test before you nest a second if. Three booleans give 8 possible rows. You do not need all 8 yet. Name the one row you want, and let the other rows fall into else.

Same numbers, two connectors.

And

n > 0 is true

m > 0 is false

And is false

Or

n > 0 is true

m > 0 is false

Or is true

Classify 4 with two tests

1. Store int n = 4. The test n > 0 is true. The test n < 10 is true.

2. n > 0 && n < 10 is true, so the if block runs. That block is the one for a small positive value.

3. Change n to 0 and repeat. n > 0 is false, so && does not need the second test, and else runs.

Problem 1. n is 3 and m is 0. What are n > 0 && m > 0 and n > 0 || m > 0?

Answer. The && test is false, because m > 0 is false. The || test is true, because n > 0 is true. One true side is enough for ||.

Problem 2. The first half of n > 0 && m > 0 is false. Does Java evaluate m > 0?

Answer. No. && stops when the left side is false, because the whole test is already false. That skip matters when the right side would have been an illegal read.

Lecture 4 · 90 minutes

A loop is an algorithm you can count

An algorithm is a finite list of steps that stops on every input you agreed to accept. A loop is the step you repeat. for (int i = 1; i <= 4; i = i + 1) runs the body 4 times. You should know that count before you run it. The update i = i + 1 is the mechanism that makes i <= 4 become false. Delete the update and the list of steps never finishes.

After each pass, name what is true. After one pass, one number is in the sum. After four passes, four numbers are in it. Start s at 0 and set s = s + i in the body. The partial sums are 1, then 3, then 6, then 10. When the test fails, s is 10, the sum of 1 through 4. That sentence is the algorithm. The for line is only how you write it.

Repeat one add, four times.

1. s starts at 0

2. Add i, then grow i

3. Stop after i passes 4

4. s holds 10

Hand trace of the four additions

1. i is 1 and s becomes 0 + 1 = 1. Then i becomes 2.

2. s becomes 1 + 2 = 3, then 3 + 3 = 6, then 6 + 4 = 10. That is four additions.

3. i becomes 5. The test i <= 4 fails. You stop with s equal to 10.

Problem 1. The loop is for (int i = 1; i <= 4; i = i + 1) and the body adds i into s, which starts at 0. What is s at the end, and how many times did the body run?

Answer. s is 10, the sum 1 + 2 + 3 + 4. The body ran 4 times. The test fails when i becomes 5.

Problem 2. You delete the update i = i + 1, and i starts at 1 with the test i <= 4. Why is this no longer an algorithm for the inputs you accepted?

Answer. The test stays true, so the step list does not finish. An algorithm has to stop. The update is what lets i pass 4.

Lecture 5 · 90 minutes

A method returns one result

A method is a named block you call. static int add(int a, int b) takes two ints and returns one int. The call add(3, 5) copies 3 into a and 5 into b. The return becomes the value of the call, which is 8. The copy is the mechanism. Setting a = 0 inside add changes the copy, not the caller's variable. The caller sees only the returned int.

Keep one job in the method. add returns a sum and does not also print. A second method can print the 8 if someone must see it. The split lets you test: call add(3, 5) and compare the return with 8. If the sum is only printed, the test has to read the screen. A method that promises an int must return an int on every path, including the path that handles 0.

Call, copy, then return.

Caller writes add(3, 5) -> Copies go into a and b -> return sends 8 back -> The call value is 8

A method that sums 1 through 4

1. static int sumThrough4() sets s to 0 and loops i from 1 to 4, adding i each time.

2. The return is 10. The names s and i exist only inside the method.

3. main stores int answer = sumThrough4(). answer is 10. main does not contain the loop.

Problem 1. add(4, 6) runs a body that sets a = 0 and then returns a + b. What is the value of the call?

Answer. 6. The parameters are copies, so a becomes 0 and b stays 6. The return is 0 + 6. The caller sees the assignment only through that returned 6.

Problem 2. Why is a method that both prints and returns the sum harder to test than a method that only returns it?

Answer. A test of the method that only returns compares the return with 8, or with 10 for the sum of 1 through 4. A method that also prints forces the test to read the screen, which mixes two jobs.

Lecture 6 · 90 minutes

An array is a numbered row

An array stores several values of one type under one name. int[] a = new int[4] creates 4 ints. Legal indexes are 0, 1, 2, and 3. a[0] = 10 writes the first slot. a.length is 4, so the last legal index is 3. Java checks the index. Reading a[4] stops the program instead of quietly using bytes past the row. You let that stop stand. Slot 4 does not exist.

A scan visits each slot once. Start s at 0 and i at 0. While i < a.length, add a[i], then add 1 to i. With values 1, 2, 3, and 4, the sum is 10, and i ends at 4, which is not a legal index. The test i < a.length keeps the read inside the row. i <= a.length tries index 4 and the program stops. The four numbers share one name. You call that scan the algorithm.

Four slots in one array.

new int[4] creates four slots

Index 0 holds the first

Index 3 is the last legal

a.length is 4

Sum the row 1, 2, 3, 4

1. Store 1, 2, 3, and 4 in a. Set s = 0 and i = 0.

2. While i < 4, add a[i] to s and add 1 to i. The indexes you read are 0, 1, 2, and 3.

3. s is 10 and i is 4. You do not read a[4]. The length told you where to stop.

Problem 1. a.length is 4. Which indexes are legal, and what does Java do if you read a[4]?

Answer. Legal indexes are 0, 1, 2, and 3, and the last is a.length - 1. Reading a[4] stops the program. Java checks the index.

Problem 2. The slots hold 1, 2, 3, and 4. The loop adds a[i] while i < a.length, and s starts at 0. What is s?

Answer. s is 10. The loop reads four slots and stops when i becomes 4, without reading past the end.

Lecture 7 · 90 minutes

A class groups fields and one action

A class is a plan for objects. class Counter { int value; } says each Counter has its own int. new Counter() creates one object. c.value = 4 stores 4 in c, not in the class. A second object d has a different value slot. Changing c does not change d. That separate field is the mechanism. Without the class you would pass loose ints and hope you remember which ones form a pair.

A method acts on one object. int bump() { value = value + 1; return value; } uses the field of the object you called. If c.value is 4, c.bump() stores 5 and returns 5. d stays as it was if you did not call bump on d. You construct, then you call. A call before new has no object, so there is no slot holding 4. One class and one returned int are enough here.

Two objects, one class.

Object c

Its own value slot

Starts at 4

bump returns 5

Object d

A different value slot

Stays at its own number

Not changed by c.bump

Bump one counter from 4 to 5

1. Counter c = new Counter(); then c.value = 4. Counter d = new Counter(); then d.value = 1.

2. int got = c.bump(). The method adds 1 to c.value and returns 5. got is 5.

3. d.value is still 1. The call named c, so only c's field was updated.

Problem 1. c.value is 4 and d.value is 1. You call c.bump(), which adds 1 and returns the field. What are c.value and d.value afterward?

Answer. c.value is 5, and the return is 5. d.value stays 1, because bump ran on c, and each object has its own field.

Problem 2. Why does calling bump require new first?

Answer. bump updates the value field of one object. new creates that object. Without it there is no slot holding 4, and no object to receive the call.

Lecture 8 · 90 minutes

You can trace a small Java program

You can now trace a short Java program by hand. Lines run in order. The int division 7 / 2 stores 3, and && can skip its second test. You can write a loop from 1 to 4 that sums to 10, a method that returns that sum, and a scan of an array of length 4 that stops at a.length. You can put an int in a class, construct one object, and return the

updated field.

Check two things before you trust the run. Trace a boolean with real numbers, including one case that takes else, such as n equal to 0. Then trace the loop and write the partial sums 1, 3, 6, and 10. If the method return and your paper disagree, the bug is the bound or the update, not the print. Fix the method. A program you cannot trace on 4 items is not finished, even when it compiles.

What you can now trace.

Lines in main, in order

A test with one false case

A loop from 1 to 4

A return you can recompute

Sum 1 through 4, then bump a counter

1. static int sumThrough4() loops i from 1 to 4, adds i into s, and returns 10.

2. main stores that return in a Counter field: c.value = sumThrough4(), so the field becomes 10.

3. c.bump() adds 1 and returns 11. Your paper trace should show 10, then 11, before you look at any printed line.

Problem 1. A loop uses i from 1 through 4. An array a of length 4 holds 1, 2, 3, and 4. If the body adds a[i] on those i values, which read is illegal, and what is the sum of the four slots at indexes 0 through 3?

Answer. a[4] is illegal. The last legal index is 3, and the loop's last i is 4. The sum of the four slots, read at indexes 0, 1, 2, and 3, is 10.

Problem 2. c.value is 4. You wanted d, a second Counter, to stay at 1 while c becomes 5. Which call does that?

Answer. c.bump(). It adds 1 to c's own field, so c.value becomes 5. d.value stays 1 because you did not call bump on d.